

Guia de Tecnologias do Projeto

Um material didatico para entender todas as principais tecnologias usadas no sistema, onde elas aparecem no codigo e como cada uma participa do funcionamento real da aplicacao.

PROJETO

Sistema web para pedidos antecipados em cantinas escolares

PASTA BASE

C:\Users\forti\Documents\digital-flavor

STACK CENTRAL

React, Vite, TypeScript, Tailwind CSS, Supabase e Vercel

GERADO EM

14 de junho de 2026

Este guia foi montado a partir dos arquivos reais do repositório: `package.json`, `src/`, `supabase/`, `tests/`, `vite.config.ts`, `playwright.config.ts` e `vercel.json`.

Sumario

1. Visao geral da arquitetura

3. Node.js, npm e scripts

5. React, componentes e estado

7. TypeScript e orientacao a objetos

9. Tailwind CSS, PostCSS e UI

11. PostgreSQL, migrations e RLS

13. Deploy, Analytics e Speed Insights

2. Estrutura de pastas

4. Vite e entrada da aplicacao

6. React Router

8. Estruturas de dados

10. Autenticacao local e Supabase

12. Supabase Edge Functions

14. Testes, qualidade e comandos

1. Visao Geral Da Arquitetura

O Digital Flavor e uma aplicacao web de pagina unica. O navegador carrega o React, o React Router escolhe a tela correta, o `App.tsx` controla o estado principal e as regras de negocio ficam nas classes de dominio. O projeto pode funcionar em modo demonstracao com `localStorage` ou integrado ao Supabase quando as variaveis de ambiente estao configuradas.



Mapa rapido das tecnologias

Tecnologia	Onde aparece	Funcao dentro do projeto
Node.js e npm	<code>package.json</code> , <code>package-lock.json</code>	Instalam dependencias e executam scripts de desenvolvimento, teste e build.
Vite	<code>vite.config.ts</code>	Servidor de desenvolvimento, empacotamento final e integracao com Vitest.
React e React DOM	<code>src/main.tsx</code> , <code>src/features/</code>	Renderizam a interface de aluno, autenticacao, conta e painel admin.
React Router DOM	<code>src/main.tsx</code> , <code>src/App.tsx</code>	Controla as paginas <code>/login</code> , <code>/cadastro</code> , <code>/admin</code> e outras rotas.
TypeScript	<code>src/**/*.ts</code> , <code>src/**/*.tsx</code> , <code>tsconfig*.json</code>	Tipa props, entidades, servicos, banco e eventos para reduzir erros.
Tailwind CSS	<code>src/styles.css</code> , classes em TSX	Cria layout responsivo, cores, espacamentos, cards, botoes e estados visuais.
PostCSS e Autoprefixer	<code>postcss.config.js</code>	Processam CSS e adicionam prefixos de compatibilidade quando necessario.
clsx	<code>src/components/ui.tsx</code>	Combina classes CSS de forma condicional em componentes reutilizaveis.
lucide-react	<code>src/features/**</code>	Forneca icones vetoriais para botoes, status, menu e paineis.
Supabase JS, Auth e PostgreSQL	<code>src/lib/</code> , <code>supabase/</code>	Autenticacao, banco relacional, policies, funcoes e sincronizacao remota.
Deno e Supabase Edge Functions	<code>supabase/functions/</code>	Executam endpoints serverless protegidos por JWT.
Vercel	<code>vercel.json</code> , <code>@vercel/*</code>	Hospeda o front-end, redireciona rotas da SPA e mede analytics/performance.
Vitest, Testing Library e JSDOM	<code>src/App.test.tsx</code> , <code>src/domain/domain.test.ts</code>	Testam componentes, regras de negocio e fluxos sem abrir navegador real.
Playwright	<code>playwright.config.ts</code> , <code>tests/e2e/</code>	Testa a aplicacao em navegadores de verdade, inclusive desktop e mobile.
ESLint e typescript-eslint	<code>eslint.config.js</code>	Fiscalizam padrao de codigo, hooks React e exportacoes seguras para refresh.

2. Estrutura De Pastas

A organizacao do repositorio separa configuracoes, codigo de interface, dominio, integracao com Supabase, testes e documentacao. Isso facilita explicar a arquitetura porque cada pasta tem responsabilidade clara.

```
digital-flavor/
|-- docs/                Material de documentacao do projeto
|-- src/
|   |-- App.tsx          Orquestracao de rotas, estado e fluxos
|   |-- main.tsx         Entrada React + BrowserRouter + Vercel metrics
|   |-- styles.css       Tailwind e variaveis globais
|   |-- auth/demoAuth.ts Autenticacao local via localStorage
|   |-- components/ui.tsx Button, Panel, StatusBadge, MetricCard
|   |-- data/seed.ts     Produtos, estoque e pedidos de demonstracao
|   |-- domain/          Classes, servicos e estruturas de dados
|   |-- features/        Telas de cliente, admin, conta, auth e layout
|   |-- lib/             Supabase client, auth remoto e Edge Functions
|   |-- theme/           Mapa de cores/status
|   |-- utils/           Formatacao de moeda, CPF e RA
|   |-- test/setup.ts    Setup dos testes unitarios
|-- supabase/
|   |-- migrations/      SQL do banco, tabelas, constraints, RLS
|   |-- functions/       Edge Functions em Deno
|   |-- config.toml      Configuracao local do Supabase
|-- tests/e2e/           Testes Playwright
|-- package.json         Dependencias e comandos npm
|-- vite.config.ts       Vite + Vitest
|-- playwright.config.ts Configuracao e2e
|-- eslint.config.js     Regras de lint
|-- vercel.json          Deploy Vercel e rewrite da SPA
```

Ideia principal: `src/features` mostra a aplicacao para o usuario, `src/domain` guarda as regras de negocio, `src/lib` conversa com servicos externos e `supabase` descreve banco, auth e funcoes remotas.

3. Node.js, npm E Scripts

Node.js é o ambiente usado para rodar ferramentas de desenvolvimento. O npm lê o `package.json`, instala pacotes e executa scripts padronizados. O projeto define comandos para desenvolvimento, build, lint, teste unitário e teste ponta a ponta.

```
{
  "scripts": {
    "dev": "vite --host 127.0.0.1",
    "build": "tsc -b && vite build",
    "lint": "eslint . --max-warnings=0",
    "test": "vitest run",
    "test:watch": "vitest",
    "test:e2e": "playwright test"
  }
}
```

Como isso funciona no nosso projeto

- `npm run dev` sobe o Vite para abrir a aplicação durante o desenvolvimento.
- `npm run build` primeiro valida TypeScript com `tsc -b` e depois gera `dist/`.
- `npm run lint` impede warnings de qualidade com `--max-warnings=0`.
- `npm run test` roda Vitest para regras e componentes.
- `npm run test:e2e` abre Playwright e executa fluxos reais no navegador.

4. Vite E Entrada Da Aplicacao

Vite é o bundler/servidor de desenvolvimento. Ele entrega recarregamento rápido em desenvolvimento e gera os arquivos estáticos finais para deploy. O projeto usa o plugin oficial de React.

```
// vite.config.ts
import react from '@vitejs/plugin-react'
import { defineConfig } from 'vite'

export default defineConfig({
  plugins: [react()],
  test: {
    css: true,
    environment: 'jsdom',
    exclude: ['tests/e2e/**', 'node_modules/**', 'dist/**'],
    globals: true,
    setupFiles: './src/test/setup.ts'
  }
})
```

Entrada no navegador

O arquivo `index.html` aponta para `/src/main.tsx`. Esse arquivo cria a raiz React, liga o roteador e registra as métricas da Vercel.

```
// src/main.tsx
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
      <Analytics />
      <SpeedInsights />
    </BrowserRouter>
  </React.StrictMode>
)
```

Por que isso importa: quase tudo que o usuário vê nasce depois desse ponto. O `BrowserRouter` envolve o app inteiro, então todas as telas conseguem usar rotas e navegação.

5. React, Componentes E Estado

React é a biblioteca que monta a interface a partir de componentes. No projeto, cada tela recebe dados por `props` e dispara callbacks para o `App.tsx`, que guarda o estado principal da aplicação.

Estado central no App

```
// src/App.tsx
const [session, setSession] = useState<AuthSession | undefined>(() => readSession())
const [products, setProducts] = useState(() => seedProducts.map(cloneProduct))
const [inventory, setInventory] = useState(() => seedInventory.map(cloneInventoryItem))
const [cart, setCart] = useState(() => new Cart())
const [queue, setQueue] = useState(() => seedOrders)
const [preparingOrders, setPreparingOrders] = useState<Order[]>([])
const [completedOrders, setCompletedOrders] = useState<typeof seedOrders>([])
```

Esses estados representam a sessão do usuário, produtos, estoque, carrinho e filas de pedidos. Quando um usuário confirma compra ou um admin chama o próximo pedido, o React atualiza o estado e redesenha a tela.

Componente de cliente

```
// src/features/customer/CustomerOrdering.tsx
interface CustomerOrderingProps {
  products: Product[]
  inventory: InventoryItem[]
  cartItems: CartItemSnapshot[]
  cartTotalCents: number
  pickupTime: string
  paymentMethod: PaymentMethod
  onAddProduct: (productId: string) => void
  onCheckout: () => void
}
```

O componente `CustomerOrdering` não cria pedido sozinho. Ele mostra cardápio, carrinho, horário de retirada e método de pagamento. Quando o usuário clica, ele chama funções recebidas do `App.tsx`.

Fluxo real do checkout


```
// src/App.tsx
function handleCheckout() {
  const stockService = new StockService(inventory.map(cloneInventoryItem))
  const checkout = new CheckoutService(stockService)
  const order = checkout.createOrder({
    cart,
    customerId: session?.email ?? 'cliente-digital-flavor',
    customerName: session?.name ?? 'Cliente Digital Flavor',
    paymentMethod,
    pickupTime
  })

  order.payment.approve()
  order.advance('queued')
  setQueue((current) => [...current, order])
  setInventory(stockService.snapshot().map(cloneInventoryItem))
  setCart(new Cart())
  setLatestOrderId(order.id)
}
```

Aluno adiciona item



Cart valida quantidade



CheckoutService cria Order

Payment e aprovado



Pedido entra na fila



Tela mostra posicao e codigo

6. React Router

React Router transforma caminhos da URL em telas. O projeto tem rotas para login, cadastro, completar cadastro, recuperar senha, perfil, pagamentos e painel admin. Também usa `Navigate` para proteger telas conforme a sessão.

```
// src/App.tsx
return (
  <Routes>
    <Route path="/" element={studentPage} />
    <Route path="/login" element={session ? <Navigate to="/" replace /> : <LoginPage ... />} />
    <Route path="/cadastro" element={session ? <Navigate to="/" replace /> : <RegisterPage ... />} />
    <Route path="/configuracoes" element={studentAccountPage(<AccountSettingsPage ... />)} />
    <Route path="/admin" element={adminPage} />
    <Route path="*" element={<Navigate to={session?.role === 'admin' ? '/admin' : '/'} replace />} />
  </Routes>
)
```

Como a protecao funciona

- Sem sessão, o usuário vai para `/login`.
- Aluno sem RA/CPF completo vai para `/completar-cadastro`.
- Admin logado vai para `/admin`.
- Rotas desconhecidas voltam para a página correta do perfil atual.

7. TypeScript E Orientacao A Objetos

TypeScript adiciona tipos ao JavaScript. No Digital Flavor, ele aparece em tres pontos fortes: tipagem das props dos componentes, contratos de dados vindos do Supabase e classes de dominio orientadas a objetos.

Classe Product

```
// src/domain/Product.ts
export class Product {
  readonly id: string
  name: string
  category: ProductCategory
  priceCents: number
  sustainabilityScore: number
  active: boolean

  constructor(props: ProductProps) {
    if (props.priceCents < 0) {
      throw new Error('Product price cannot be negative')
    }

    this.id = props.id
    this.name = props.name
    this.category = props.category
    this.priceCents = props.priceCents
    this.sustainabilityScore = Math.max(0, Math.min(100, props.sustainabilityScore))
    this.active = props.active ?? true
  }

  updatePrice(priceCents: number) {
    if (priceCents < 0) throw new Error('Product price cannot be negative')
    this.priceCents = priceCents
  }
}
```

Aqui a classe protege regras do produto: preco nao pode ser negativo e pontuacao de sustentabilidade fica entre 0 e 100. A tela admin chama metodos como `updatePrice`, `activate` e `deactivate` em vez de alterar tudo solto.

Classe Cart

```
// src/domain/Cart.ts
export class Cart {
  private items = new Map<string, CartItemSnapshot>()

  addProduct(product: Product, quantity: number, stock?: InventoryItem) {
    this.assertQuantity(quantity)
    const currentQuantity = this.items.get(product.id)?.quantity ?? 0
    const nextQuantity = currentQuantity + quantity

    if (stock && nextQuantity > stock.availableQuantity) {
      throw new Error('Cart quantity exceeds available stock')
    }

    this.items.set(product.id, this.snapshot(product, nextQuantity))
  }

  get totalCents() {
    return this.listItems().reduce((sum, item) => sum + item.totalCents, 0)
  }
}
```

O `Cart` usa `Map` para indexar itens por produto. Isso evita duplicar o mesmo produto varias vezes e facilita atualizar quantidade e calcular total.

Tipos do banco

```
// src/lib/database.types.ts
export interface Database {
  public: {
    Tables: {
      products: {
        Row: {
          id: string
          name: string
          category: 'lanche' | 'bebida' | 'fruta' | 'combo'
          price_cents: number
          active: boolean
        }
      }
    }
  }
}
```

O cliente Supabase usa `createClient<Database>`. Assim, consultas remotas ficam alinhadas com as tabelas, colunas e enums que existem no PostgreSQL.

8. Estruturas De Dados

Alem das bibliotecas externas, o projeto implementa estruturas de dados pedidas no requisito avaliativo: lista, fila FIFO e pilha. Elas estao em `src/domain/structures.ts` e nos servicos de dominio.

Fila FIFO

```
// src/domain/structures.ts
export class Queue<T> {
  private items: T[] = []

  enqueue(item: T) {
    this.items.push(item)
  }

  dequeue() {
    return this.items.shift()
  }

  peek() {
    return this.items[0]
  }
}
```

FIFO significa "first in, first out": o primeiro pedido que entra e o primeiro a ser chamado para preparo. No painel admin, isso aparece quando a cantina clica em **Chamar proximo**.

Pilha LIFO

```
// src/domain/structures.ts
export class Stack<T> {
  private items: T[] = []

  push(item: T) {
    this.items.push(item)
  }

  pop() {
    return this.items.pop()
  }
}
```

LIFO significa "last in, first out": a ultima alteracao admin e a primeira a ser desfeita. A classe `AdminActionHistory` usa essa pilha para implementar **Desfazer ultima alteracao**.

Servicos que usam as estruturas

```
// src/domain/services.ts
export class OrderQueueService {
  private queue = new Queue<Order>()

  enqueue(order: Order) {
    order.advance('queued')
    this.queue.enqueue(order)
  }

  nextForPreparation() {
    const order = this.queue.dequeue()
    if (order) order.advance('preparing')
    return order
  }
}

export class AdminActionHistory {
  private stack = new Stack<AdminAction>()

  record(action: AdminAction) {
    this.stack.push(action)
  }

  undoLast() {
    const action = this.stack.pop()
    if (action) action.undo()
    return action
  }
}
```

9. Tailwind CSS, PostCSS E UI

Tailwind CSS permite estilizar diretamente no JSX com classes utilitarias. O projeto usa essa abordagem para criar telas responsivas de aluno e admin sem arquivos CSS separados para cada componente.

Config do Tailwind

```
// tailwind.config.ts
export default {
  content: ['./index.html', './src/**/*.ts,tsx'],
  theme: {
    extend: {}
  },
  plugins: []
} satisfies Config
```

A propriedade `content` diz ao Tailwind onde procurar classes. Por isso, classes escritas em `src/features/**/*.tsx` entram no CSS final.

CSS global

```
// src/styles.css
@tailwind base;
@tailwind components;
@tailwind utilities;

:root {
  --color-brand-green: #16a34a;
  --color-brand-blue: #2563eb;
  --color-brand-orange: #f97316;
  --color-brand-red: #dc2626;
  font-family: Inter, ui-sans-serif, system-ui, sans-serif;
}
```

Componente Button com clsx

```
// src/components/ui.tsx
const buttonVariantClass = {
  primary: 'bg-orange-500 text-white hover:bg-orange-600',
  secondary: 'bg-blue-600 text-white hover:bg-blue-700',
  quiet: 'border border-slate-200 bg-white text-slate-700',
  danger: 'bg-red-600 text-white hover:bg-red-700'
}

export function Button({ className, variant = 'primary', ...props }) {
  return (
    <button
      className={clsx(
        'inline-flex min-h-10 items-center justify-center gap-2 rounded-md px-4',
        buttonVariantClass[variant],
        className
      )}
      {...props}
    />
  )
}
```

`clsx` junta classes fixas, classes por variante e classes extras recebidas pelo componente. Isso evita repetir o mesmo conjunto de estilos em todos os botoes.

lucide-react

```
// src/features/customer/CustomerOrdering.tsx
import { Clock, Minus, Plus, ShoppingBag, WalletCards } from 'lucide-react'

<Button type="button" disabled={cartItems.length === 0} onClick={onCheckout}>
  <WalletCards size={18} aria-hidden="true" />
  Confirmar pedido
</Button>
```

A biblioteca `lucide-react` fornece ícones prontos como componentes React. Eles melhoram a leitura visual de ações como adicionar, pagar, voltar, salvar e acompanhar status.

PostCSS e Autoprefixer

```
// postcss.config.js
export default {
  plugins: {
    tailwindcss: {},
    autoprefixer: {}
  }
}
```

PostCSS executa plugins sobre o CSS. O Tailwind gera as classes utilitarias e o Autoprefixer adiciona prefixos de compatibilidade para navegadores quando necessario.

10. Autenticacao Local E Supabase

O projeto tem duas formas de autenticacao. Em modo demonstracao, ele usa `localStorage` no navegador. Em ambiente com variaveis de Supabase, ele usa `@supabase/supabase-js` para login com senha, Google OAuth, redefinicao de senha e sessao persistente.

Cliente Supabase

```
// src/lib/supabase.ts
const supabaseUrl = import.meta.env.VITE_SUPABASE_URL
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY

export const isSupabaseConfigured = Boolean(supabaseUrl && supabaseAnonKey)

export const supabase = isSupabaseConfigured
  ? createClient<Database>(supabaseUrl, supabaseAnonKey, {
    auth: {
      autoRefreshToken: true,
      persistSession: true,
      detectSessionInUrl: true
    }
  })
  : null
```

As variaveis `VITE_SUPABASE_URL` e `VITE_SUPABASE_ANON_KEY` ficam no ambiente. Se nao existirem, `supabase` vira `null` e a aplicacao continua funcionando localmente.

Auth local com localStorage

```
// src/auth/demoAuth.ts
const studentsKey = 'digital-flavor-students'
const sessionKey = 'digital-flavor-session'

export function registerStudentAccount(account: StudentAccount) {
  const email = normalizeEmail(account.email)
  const existing = readStudentAccounts().filter((student) => student.email !== email)
  const nextAccount = { ...account, email, name: account.name.trim() }

  if (canUseStorage()) {
    window.localStorage.setItem(studentsKey, JSON.stringify([...existing, nextAccount]))
  }

  return nextAccount
}
```

Esse modo facilita apresentacao e teste sem depender de internet ou banco configurado. Ele guarda contas, sessao, preferencias e metodos de pagamento no navegador.

Login remoto com Supabase Auth

```
// src/lib/auth.ts
export async function signInWithPassword(email: string, password: string) {
  if (!supabase) {
    throw new Error('Supabase is not configured')
  }

  return supabase.auth.signInWithPassword({ email, password })
}

export async function signInWithGoogle() {
  return supabase.auth.signInWithOAuth({
    provider: 'google',
    options: {
      redirectTo: `${window.location.origin}/auth/callback`
    }
  })
}
```

Como o App escolhe o modo

```
// src/App.tsx
const shouldUseSupabaseCustomerPasswordAuth =
  isSupabaseConfigured &&
  (import.meta.env.PROD || import.meta.env.VITE_SUPABASE_PASSWORD_AUTH === 'true')
```

- Admin de demonstracao usa credencial local `admin@digitalflavor.com`.
- Aluno pode cadastrar no modo local sem Supabase.
- Em producao ou com flag especifica, aluno usa Supabase Auth por senha.
- Login Google sempre depende de Supabase configurado.

11. PostgreSQL, Migrations E Row Level Security

O Supabase usa PostgreSQL como banco relacional. As migrations em `supabase/migrations` criam tipos, tabelas, constraints, indices, dados iniciais e policias de seguranca.

Tipos e tabelas principais

```
// supabase/migrations/20260604225344_initial_digital_flavor_schema.sql
create type public.app_role as enum ('customer', 'employee', 'manager', 'admin');
create type public.product_category as enum ('lanche', 'bebida', 'fruta', 'combo');
create type public.order_status as enum (
  'draft', 'submitted', 'queued', 'preparing', 'ready', 'completed', 'cancelled'
);
create type public.payment_method as enum ('pix', 'card', 'cash');

create table public.products (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  category public.product_category not null,
  price_cents integer not null check (price_cents >= 0),
  active boolean not null default true
);
```

Modelo de dados

Tabela	Representa	Relacao com o sistema
profiles	Usuarios e perfis	Define aluno, funcionario, gerente ou admin.
products	Cardapio	Mostra produtos para aluno e permite gestao pelo admin.
inventory	Estoque	Controla quantidade, reservas, ponto de reposicao e validade.
orders	Pedidos	Guarda status, cliente, horario, codigo de retirada e total.
order_items	Itens do pedido	Relaciona pedido com produtos comprados.
payments	Pagamentos	Registra metodo, status e valor.
stock_movements	Movimentacao de estoque	Audita reserva, consumo, reposicao e ajustes.
customer_preferences	Preferencias do aluno	Guarda turno, sala, horario padrao e notificacoes.
customer_payment_methods	Metodos de pagamento	Guarda PIX, cartao ou dinheiro preferido do cliente.

Row Level Security

RLS e a seguranca por linha do PostgreSQL. Ela impede que um usuario leia ou altere dados que nao pertencem a ele, a menos que tenha perfil de equipe/admin.

```
// Exemplo de policy
create policy "orders_select_owner_or_staff"
on public.orders
for select
to authenticated
using (
  customer_id = (select auth.uid())
  or exists (
    select 1 from public.profiles staff_profile
    where staff_profile.id = (select auth.uid())
    and staff_profile.role in ('employee', 'manager', 'admin')
  )
);
```

Na pratica: o aluno ve os proprios pedidos. Funcionarios, gerentes e admins podem ver pedidos para operar a fila da cantina.

Constraints de RA e CPF

```
// supabase/migrations/20260607202957_add_student_documents.sql
alter table public.profiles
  add column if not exists student_ra text,
  add column if not exists cpf text;

alter table public.profiles
  add constraint profiles_student_ra_format
  check (student_ra is null or student_ra ~ '^[0-9]{8}-[0-9]$');
```

A regra de documento existe em duas camadas: a interface valida RA/CPF antes de cadastrar, e o banco tambem protege o formato quando os dados chegam ao Supabase.

12. Supabase Edge Functions

Edge Functions são funções serverless do Supabase. Elas rodam em Deno, recebem requisições HTTP e podem usar o token do usuário para consultar o Supabase de forma segura.

Invocação no front-end

```
// src/lib/edgeFunctions.ts
export async function saveCustomerExperience(payload: CustomerExperiencePayload) {
  if (!supabase) {
    return { synced: false, reason: 'Supabase is not configured' }
  }

  const { error } = await supabase.functions.invoke('customer-preferences', {
    body: payload
  })

  if (error) {
    return { synced: false, reason: error.message }
  }

  return { synced: true }
}
```

Função customer-preferences

```
// supabase/functions/customer-preferences/index.ts
Deno.serve(async (req) => {
  if (req.method === 'OPTIONS') {
    return new Response('ok', { headers: corsHeaders })
  }

  const supabase = getSupabaseClient(req)
  const { data: { user }, error: userError } = await supabase.auth.getUser()

  if (userError || !user) {
    return jsonResponse({ error: 'Sessão inválida.' }, 401)
  }

  const body = await req.json()
  await supabase.from('customer_preferences').upsert({
    profile_id: user.id,
    phone: body.profile?.phone ?? '',
    default_pickup_time: body.preferences?.defaultPickupTime ?? '10:30'
  })

  return jsonResponse({ ok: true })
})
```

CORS compartilhado

```
// supabase/functions/_shared/cors.ts
export const corsHeaders = {
  'Access-Control-Allow-Origin': '*',
  'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',
  'Access-Control-Allow-Methods': 'GET, POST, OPTIONS'
}
```

O CORS permite que o front-end hospedado na Vercel chame as funcoes do Supabase. O projeto tambem configura `verify_jwt = true` em `supabase/config.toml`, exigindo sessao valida.

13. Deploy, Analytics E Speed Insights

O projeto esta preparado para deploy na Vercel. Como e uma SPA, qualquer rota do navegador precisa voltar para `index.html`, e o React Router decide qual tela exibir.

Configuracao da Vercel

```
// vercel.json
{
  "framework": "vite",
  "installCommand": "npm install",
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

Metricas no React

```
// src/main.tsx
import { Analytics } from '@vercel/analytics/react'
import { SpeedInsights } from '@vercel/speed-insights/react'

<BrowserRouter>
  <App />
  <Analytics />
  <SpeedInsights />
</BrowserRouter>
```

- `@vercel/analytics` coleta informacoes de uso e acesso.
- `@vercel/speed-insights` mede indicadores de performance percebidos pelo usuario.
- `rewrites` evita erro 404 quando o usuario abre diretamente `/admin` ou `/login`.
- As variaveis `VITE_SUPABASE_URL` e `VITE_SUPABASE_ANON_KEY` devem existir no ambiente da Vercel para o Supabase remoto funcionar.

14. Testes, Qualidade E Comandos

O projeto usa camadas diferentes de validacao: TypeScript para tipos, ESLint para padrao de codigo, Vitest para testes de componentes/regras e Playwright para fluxo real no navegador.

Vitest + Testing Library + user-event

```
// src/App.test.tsx
import { render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { MemoryRouter } from 'react-router-dom'
import { describe, expect, it } from 'vitest'

it('registers a client, creates an order, and shows the client position', async () => {
  const user = userEvent.setup()
  render(
    <MemoryRouter initialEntries={['/cadastro']}>
      <App />
    </MemoryRouter>
  )

  await user.type(screen.getByLabelText(/Nome completo/i), 'Aluno Teste')
  await user.click(screen.getByRole('button', { name: /Cadastrar e entrar/i }))
  expect(await screen.findByRole('heading', { name: /Cardapio para o intervalo/i }))
    .toBeInTheDocument()
})
```

`Testing Library` testa a tela como o usuario enxerga: labels, botoes e textos. `user-event` simula digitacao e clique. `jest-dom` adiciona matchers como `toBeInTheDocument()`. `JSDOM` cria um DOM falso para rodar esses testes no Node.

Playwright

```
// playwright.config.ts
export default defineConfig({
  testDir: './tests/e2e',
  use: {
    baseURL: 'http://127.0.0.1:5173',
    trace: 'on-first-retry'
  },
  webServer: {
    command: 'npm run dev -- --port 5173',
    url: 'http://127.0.0.1:5173',
    reuseExistingServer: !process.env.CI
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'mobile-chrome', use: { ...devices['Pixel 7'] } }
  ]
})
```


Playwright abre a aplicacao de verdade, preenche formulario, clica em botoes e valida a navegacao. O projeto testa fluxo de aluno e fluxo de admin em desktop e mobile.

ESLint

```
// eslint.config.js
export default tseslint.config(
  { ignores: ['dist', 'coverage', 'playwright-report', 'node_modules'] },
  js.configs.recommended,
  ...tseslint.configs.recommended,
  {
    files: ['**/*.ts', '**/*.tsx'],
    plugins: {
      'react-hooks': reactHooks,
      'react-refresh': reactRefresh
    },
    rules: {
      ...reactHooks.configs.recommended.rules,
      'react-refresh/only-export-components': ['warn', { allowConstantExport: true }]
    }
  }
)
```

Comandos para validar

Comando	O que valida
npm run lint	Padrao de codigo, regras TypeScript, regras de hooks React e refresh.
npm run test	Testes unitarios/de componentes no Vitest.
npm run build	Compilacao TypeScript e bundle Vite final em dist/ .
npm run test:e2e	Fluxos reais de usuario no Playwright.

Resumo Para Explicar O Projeto

O Digital Flavor usa React para interface, React Router para navegacao, TypeScript para seguranca de tipos, classes de dominio para regras de negocio, Tailwind para visual responsivo, Supabase para autenticacao, banco PostgreSQL e funcoes remotas, Vercel para deploy e ferramentas de teste para garantir que aluno e admin consigam cumprir os principais fluxos.

Aluno Entra ou cadastra, completa RA/CPF, escolhe itens, define horario e pagamento, confirma pedido e acompanha posicao.	Admin Entra no painel, chama pedidos por ordem de chegada, marca preparo, entrega, edita estoque, preco e disponibilidade.
Dominio Classes como <code>Product</code> , <code>Cart</code> , <code>Order</code> , <code>Payment</code> e <code>InventoryItem</code> concentram regras.	Dados Modo demo usa <code>localStorage</code> . Modo remoto usa Supabase Auth, PostgreSQL, RLS e Edge Functions.

Frase final pronta

"Nosso projeto combina uma interface moderna em React com regras de negocio bem separadas em TypeScript, banco seguro no Supabase e deploy na Vercel. Assim, o aluno faz o pedido antecipado, a cantina organiza a fila e o estoque, e os testes garantem que os fluxos principais continuem funcionando."